

72.60 - Programación Funcional - Trabajo Final

Grafos Concurrentes en Haskell

Graphaskell



Julio 2026

Cristian Nicolás Tepedino - Legajo 62830

1. Introducción.....	3
2. Contexto y Problema.....	4
2.1. Ejemplo de ejecución del modelo Pregel.....	5
3. Solución Propuesta.....	10
3.1. Formato de Interacción.....	10
3.2. Algoritmos implementados.....	11
3.3. Principios de Diseño.....	12
4. Detalles de Diseño.....	14
4.1. Arquitectura del Sistema.....	14
4.2. Estructura del Proyecto.....	15
4.3. Modelo de Grafos.....	17
4.4. Modelo de Pregel y Manejo de Concurrencia.....	20
4.5. Modelo de Algoritmos y Extensibilidad.....	23
4.6. Manejo de Errores.....	25
4.7. Manejo de Trazas.....	26
5. Metodología de Pruebas.....	28
5.1. Motor Secuencial Como Referencia de Validación.....	28
5.2. Verificación Basada en Propiedades.....	29
5.3 Validación de la salida de la aplicación.....	29
6. Ejemplos de ejecución.....	30
6.1 BFS.....	30
6.2. Bellman-Ford.....	31
6.3 PageRank.....	31
6.4 Componentes Conexas.....	32
6.5 Label Propagation.....	33
7. Cambios Realizados Respecto a la Propuesta Inicial y Limitaciones de Diseño.....	34
7.1. Aclaración Sobre Uso de Inteligencia Artificial en el Desarrollo.....	34
8. Conclusiones y Trabajo Futuro.....	36
9. Bibliografía.....	37

1. Introducción

El presente informe describe el trabajo desarrollado para la evaluación final de la materia 72.60 - Programación Funcional. El objetivo principal consiste en el diseño e implementación de una aplicación en Haskell que incorpore interacción con el entorno y permita demostrar la aplicación práctica de los conceptos estudiados durante la cursada, en particular aquellos relacionados con programación funcional, manejo de efectos mediante mónadas y concurrencia.

La propuesta desarrollada consiste en un framework de algoritmos sobre grafos basado en el modelo de procesamiento concurrente Pregel, adaptado a un entorno funcional. En este modelo, cada vértice del grafo actúa como una unidad independiente de cómputo que procesa mensajes, actualiza su estado local y se comunica con sus vecinos a través de rondas de ejecución denominadas supersteps. Esta aproximación permite estudiar cómo problemas tradicionalmente asociados a sistemas distribuidos y procesamiento paralelo pueden expresarse utilizando técnicas propias de la programación funcional.

Uno de los principales desafíos del proyecto consiste en modelar la concurrencia preservando las propiedades de inmutabilidad y composición características del paradigma funcional. Para ello se emplean las bibliotecas `async` y `STM` sobre el runtime de threads de GHC, permitiendo coordinar el intercambio de mensajes entre vértices de manera segura y declarativa.

Asimismo, el trabajo utiliza distintas mónadas para encapsular y organizar los efectos presentes en la aplicación. En particular, se emplean abstracciones para representar el estado interno de los vértices, registrar trazas de ejecución, gestionar operaciones de entrada y salida y modelar situaciones en las que un resultado puede no existir. De esta manera, el proyecto busca ilustrar cómo la programación funcional permite separar la lógica del dominio de los efectos asociados a la ejecución del sistema.

El sistema desarrollado permite definir grafos, ejecutar algoritmos de exploración y búsqueda de caminos sobre ellos, visualizar la evolución del procesamiento por rondas y obtener los resultados finales mediante una aplicación de consola (CLI). A través de este caso de estudio se pretende demostrar la aplicación integrada de conceptos fundamentales de programación funcional, concurrencia y modelado de efectos en un problema de complejidad moderada y relevancia práctica.

2. Contexto y Problema

Los grafos constituyen una estructura matemática ampliamente utilizada para representar relaciones entre entidades. Formalmente, un grafo está compuesto por un conjunto de vértices y un conjunto de aristas que conectan pares de vértices. Esta representación permite modelar una gran variedad de sistemas reales, tales como redes de transporte, redes de comunicación, redes sociales, sistemas logísticos y mapas de navegación.

Sobre estas estructuras se ejecutan numerosos algoritmos destinados a resolver problemas de conectividad, búsqueda de caminos, cálculo de distancias mínimas, detección de componentes conexas y análisis de centralidad, entre otros. A medida que el tamaño de los grafos aumenta, también crece la complejidad computacional de estos algoritmos, haciendo necesario el empleo de técnicas que permitan distribuir la carga de procesamiento entre múltiples unidades de cómputo.

Con el objetivo de abordar este problema, Google presentó Pregel, un modelo de procesamiento orientado específicamente a la ejecución de algoritmos sobre grafos de gran escala. La propuesta se basa en el paradigma denominado *vertex-centric computing*, en el cual el procesamiento se organiza alrededor de los vértices del grafo en lugar de realizarse mediante una visión global de toda la estructura.

En Pregel, cada vértice posee un estado local y una función de cómputo asociada. La ejecución se organiza en iteraciones sincronizadas denominadas *supersteps*. Durante cada superstep, un vértice recibe los mensajes enviados por otros vértices en la ronda anterior, procesa dicha información, actualiza su estado interno y, si corresponde, genera nuevos mensajes dirigidos a sus vecinos. Estos mensajes serán entregados y procesados en el siguiente superstep.

La comunicación entre vértices constituye el mecanismo principal mediante el cual la información se propaga a través del grafo. Este enfoque permite expresar de manera natural numerosos algoritmos clásicos. Por ejemplo, en el cálculo de caminos mínimos, cada vértice puede propagar a sus vecinos la mejor distancia conocida hasta el momento. De manera similar, algoritmos de búsqueda en anchura pueden implementarse propagando niveles de exploración entre los vértices conectados.

Una característica importante del modelo es la sincronización global entre supersteps. Todos los vértices completan su procesamiento antes de que comience la siguiente ronda, garantizando que los mensajes producidos durante una iteración sean visibles únicamente en la iteración posterior. Este mecanismo simplifica el razonamiento sobre el comportamiento del sistema y evita diversos problemas asociados a la ejecución concurrente no coordinada.

Asimismo, Pregel incorpora un mecanismo de activación y desactivación de vértices. Cuando un vértice considera que no tiene más trabajo por realizar, puede pasar a un estado inactivo. Sin embargo, si recibe un nuevo mensaje en una ronda posterior, vuelve a activarse automáticamente. La ejecución completa finaliza cuando todos los vértices se encuentran inactivos y no existen mensajes pendientes de entrega.

Gracias a estas características, Pregel proporciona un modelo simple y escalable para la ejecución de algoritmos sobre grafos, permitiendo expresar problemas complejos mediante la interacción local entre vértices. Este trabajo toma dicho modelo como referencia conceptual para desarrollar una implementación capaz de ejecutar algoritmos de exploración y búsqueda de caminos sobre grafos, reproduciendo sus principales mecanismos de procesamiento y comunicación.

Es importante destacar que Pregel no implementa directamente un algoritmo de búsqueda de caminos. El modelo define únicamente la forma en que los vértices procesan información y se comunican entre sí. El comportamiento concreto surge de la función de cómputo asociada a cada vértice y de las condiciones iniciales de ejecución.

Desde una perspectiva conceptual, el modelo Pregel puede describirse mediante una función de cómputo ejecutada por cada vértice durante cada superstep:

$$\text{compute}(\text{vertexState}, \text{incomingMessages}) \rightarrow (\text{newState}, \text{outgoingMessages})$$

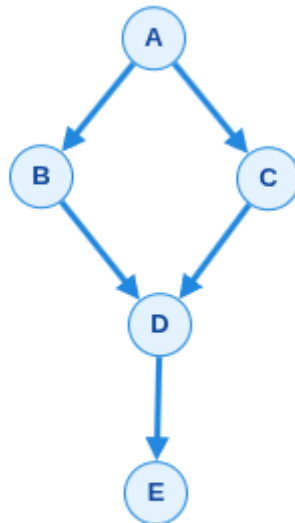
Donde:

- `vertexState` representa el estado local almacenado por el vértice.
- `incomingMessages` es el conjunto de mensajes recibidos durante el superstep actual.
- `newState` es el nuevo estado resultante luego del procesamiento.
- `outgoingMessages` es el conjunto de mensajes que serán enviados a otros vértices y procesados en el siguiente superstep.

El motor de ejecución de Pregel es independiente de la implementación concreta de esta función. Su responsabilidad consiste únicamente en coordinar la entrega de mensajes, la ejecución sincronizada de los supersteps y la activación o desactivación de los vértices. De esta manera, diferentes algoritmos pueden implementarse definiendo distintas funciones de cómputo sobre una misma infraestructura de ejecución.

2.1. Ejemplo de ejecución del modelo Pregel

Para ilustrar el funcionamiento del modelo Pregel, considérese el siguiente grafo dirigido, compuesto por los vértices A, B, C, D y E



El objetivo es calcular la distancia mínima desde el vértice A hacia el resto de los vértices utilizando una estrategia similar a una búsqueda en anchura (*Breadth-First Search*). Cada vértice almacena la menor distancia conocida desde el nodo origen y, cuando descubre una distancia más corta, propaga dicha información a sus vecinos.

Conceptualmente, la lógica de cómputo puede expresarse de la siguiente manera:

compute(distance, messages):

```
    bestDistance = minimum(messages)
    if (bestDistance < distance) {
        distance = bestDistance
        sendMessageToNeighbours(bestDistance + 1)
    }
}
```

Esta definición permite implementar una búsqueda en anchura utilizando exclusivamente comunicación local entre vértices, sin necesidad de que exista una entidad central que controle la exploración del grafo.

Como condición inicial, el vértice A almacena la distancia 0 y se considera activo al comenzar la ejecución. Todos los demás vértices almacenan distancia infinita y permanecen inactivos hasta recibir un mensaje.

Superstep 0:

El vértice A se encuentra activo y envía a sus vecinos B y C un mensaje indicando que pueden alcanzarse con distancia 1.

Vértice	Distancia
A	0
B	∞
C	∞
D	∞
E	∞

Mensajes enviados:

A -> B: 1

A -> C: 1

Superstep 1:

Los vértices B y C reciben los mensajes enviados por A. Ambos procesan el mensaje de forma concurrente.

Ambos actualizan su distancia a 1 y propagan la información hacia D indicando una distancia acumulada de 2.

Vértice	Distancia
A	0
B	1
C	1
D	∞
E	∞

B -> D: 2

C -> D: 2

Superstep 2:

El vértice D recibe dos mensajes con el mismo valor (2).

Como ambos representan una mejora respecto de su distancia actual, D actualiza su estado a distancia 2 y envía un mensaje a E con distancia 3.

Vértice	Distancia
A	0

B	1
C	1
D	2
E	∞

Mensajes enviados:

D -> E : 3

Superstep 3:

El vértice E recibe el mensaje enviado por D y actualiza su distancia a 3.

Vértice	Distancia
A	0
B	1
C	1
D	2
E	3

No se generan nuevos mensajes.

Finalización

Al finalizar el superstep 3, no existen mensajes pendientes y todos los vértices permanecen inactivos. En consecuencia, el algoritmo termina.

Las distancias mínimas obtenidas desde el vértice A son almacenadas en cada vértice.

Vértice	Distancia
A	0
B	1
C	1
D	2
E	3

Este ejemplo permite observar los conceptos fundamentales del modelo Pregel: procesamiento local en cada vértice, intercambio de mensajes, ejecución por supersteps sincronizados y propagación gradual de información a través del grafo. Aunque el ejemplo

utiliza un grafo pequeño, el mismo mecanismo puede aplicarse sobre grafos de millones de vértices, manteniendo la misma lógica de cómputo distribuido.

El resultado obtenido no depende de una lógica específica incorporada al motor de ejecución de Pregel. El motor simplemente coordina el intercambio de mensajes y la ejecución por supersteps. En este caso, la función de cómputo implementa una estrategia de búsqueda en anchura, pero el mismo mecanismo podría utilizarse para resolver otros problemas sobre grafos, como caminos mínimos ponderados, cálculo de componentes conexas, propagación de etiquetas o algoritmos de ranking de vértices.

3. Solución Propuesta

La solución implementada fue denominada “Graphaskell”. Consiste en una aplicación de consola implementada en Haskell que permite cargar un grafo desde un archivo de texto, ejecutar algoritmos de procesamiento de grafos sobre un motor de cómputo inspirado en el modelo Pregel y presentar los resultados obtenidos junto con información de la ejecución organizada por supersteps.

A diferencia de las implementaciones tradicionales de Pregel, que operan sobre clusters distribuidos, Graphaskell adapta los principios fundamentales del modelo (vértices como unidades independientes de cómputo, intercambio de mensajes y ejecución sincronizada por rondas) a un entorno concurrente local ejecutado sobre un único proceso. Para ello, se utilizan mecanismos de concurrencia provistos por GHC, principalmente threads coordinados mediante async y estructuras de comunicación implementadas con STM.

La arquitectura del sistema separa explícitamente la lógica de dominio de los mecanismos de ejecución concurrente. Mientras que la definición de los algoritmos se mantiene en funciones puras, la gestión de efectos, entrada/salida y sincronización queda encapsulada dentro del motor de ejecución. Esta organización permite aplicar principios de programación funcional manteniendo al mismo tiempo la interacción con el entorno requerida por la aplicación.

3.1. Formato de Interacción

Graphaskell es una aplicación de consola (CLI). La ejecución se realiza indicando la ubicación del archivo que contiene el grafo y los parámetros necesarios para el algoritmo seleccionado. Dependiendo del algoritmo a ejecutar, puede ser necesario especificar un vértice origen y/o un vértice destino.

Los posibles parámetros de ejecución se listan en la siguiente tabla:

Flag	Shorthand	Descripción
–graph	-g	Path al archivo de grafo
–source	-s	Vértice origen
–target	-t	Vértice destino
–algorithm	-a	Algoritmo a ejecutar
–threads		Cantidad de Threads
–verbose	-v	Trazas Detalladas

Dado que no existe un formato de archivo estandarizado para la definición de grafos, se definió un formato propio para la carga de datos. Las directivas soportadas por el formato se describen en la siguiente tabla:

Directiva	Descripción
NODES <n>	Declara que el grafo tiene n nodos, con n siendo un entero positivo. Obligatoria
WEIGHTED	Indica que las aristas del grafo tienen peso (Grafo ponderado). Opcional
EDGES	Marca el inicio de la sección de aristas. Las líneas siguientes se interpretan como aristas hasta el fin del archivo. Obligatoria

Cada directiva debe ubicarse en una línea independiente. Luego de la directiva EDGES, cada línea representa una arista utilizando el siguiente formato:

<nodo origen> <nodo destino> [peso de arista]

]donde el peso es obligatorio en grafos ponderados.

3.2. Algoritmos implementados

Con el objetivo de validar la flexibilidad del modelo de ejecución propuesto, se implementaron cinco algoritmos representativos del procesamiento de grafos. Todos ellos comparten una misma estructura de cómputo basada en vértices que mantienen un estado local, intercambian mensajes con sus vecinos y actualizan dicho estado en sucesivos supersteps hasta alcanzar una condición de convergencia.

Si bien la versión actual del sistema incluye únicamente los algoritmos descritos en esta sección, la arquitectura fue diseñada para facilitar la incorporación de nuevos algoritmos compatibles con el modelo Pregel.

3.2.1 Breadth-First Search (BFS)

El objetivo de este algoritmo es, dado un grafo no ponderado y un par origen-destino, encontrar un camino con la menor cantidad de aristas entre ambos, si existe.

La idea es explorar el grafo en anchura, por capas: primero, todos los vecinos del origen (distancia 1), luego, los vecinos de esos nodos (distancia 2) y así sucesivamente. La primera vez que se alcanza el destino se obtiene un camino óptimo en saltos. Si se agotan las capas sin visitar el destino, no hay camino.

3.2.2 Bellman-Ford

Dado un grafo ponderado y un par origen-destino, se busca hallar un camino de costo total mínimo (Single-Source Shortest Path, SSSP).

Su funcionamiento se basa en la relajación iterativa de aristas: si llega al vértice (v) un camino de costo (d) desde el origen, y existe una arista ((v, w)) de peso (c), entonces (d + c) es un candidato a distancia mínima hacia (w). Se conserva el menor valor conocido para cada vértice. El proceso continúa hasta que no se producen mejoras adicionales.

3.2.3 PageRank

Busca asignar a cada vértice de un grafo dirigido un puntaje de importancia relativo, inspirado en el ranking de páginas web: un nodo es relevante si es enlazado por otros nodos relevantes.

El algoritmo modela el comportamiento de un usuario que recorre el grafo siguiendo enlaces de forma aleatoria. En cada paso, con probabilidad (d) (factor de amortiguación, típicamente 0,85) sigue un enlace saliente al azar, y con probabilidad $(1-d)$ “teletransporta” a un nodo cualquiera del grafo. El PageRank de un vértice es la probabilidad estacionaria de estar en ese nodo bajo ese proceso. En cada iteración, el rank de un vértice se actualiza combinando la contribución de los ranks de sus predecesores (normalizados por su grado saliente) más el término de teletransporte uniforme.

Converge en grafos fuertemente conexos o con el término de amortiguación; en la práctica se itera hasta que los ranks cambian poco entre rondas.

3.2.4 Componentes Conexas (CC)

El objetivo es identificar las componentes conexas presentes en el grafo, asignando a cada vértice un identificador común para todos los vértices pertenecientes al mismo componente.

La idea es que cada vértice comienza con una etiqueta (por ejemplo, su propio identificador). En cada ronda, intercambia etiquetas con sus vecinos y adopta la menor etiqueta conocida. Si un vecino pertenece a la misma componente, su etiqueta mínima se propaga; al estabilizarse, todos los nodos de la componente comparten la misma etiqueta.

3.2.5 Label Propagation (LP)

Label Propagation es un algoritmo heurístico utilizado para la detección de comunidades en grafos.

Cada vértice comienza asociado a una etiqueta única. En cada iteración, actualiza su etiqueta adoptando aquella que aparece con mayor frecuencia entre sus vecinos. Cuando existen empates, se utiliza un criterio determinístico basado en el identificador de la etiqueta.

El proceso continúa hasta alcanzar un estado estable o hasta que la cantidad de cambios entre iteraciones resulte insignificante. El resultado final consiste en una partición aproximada del grafo en comunidades caracterizadas por una alta densidad de conexiones internas.

3.3. Principios de Diseño

La solución desarrollada busca seguir los siguientes tres principios de diseño.

Pureza funcional en el núcleo:

La lógica de negocio se expresa mediante funciones puras, sin efectos secundarios. Esto permite razonar sobre el comportamiento del sistema de forma declarativa, facilita las pruebas y mejora la predictibilidad del código.

Abstracción del modelo de cómputo:

Los algoritmos se describen a través de una abstracción común denominada AlgorithmSpec, que define las operaciones necesarias para su ejecución: inicialización del estado, generación de mensajes iniciales, actualización por superstep y extracción del resultado final.

El motor de ejecución interactúa exclusivamente con esta interfaz, lo que permite incorporar nuevos algoritmos sin modificar la infraestructura subyacente.

Encapsulamiento de efectos:

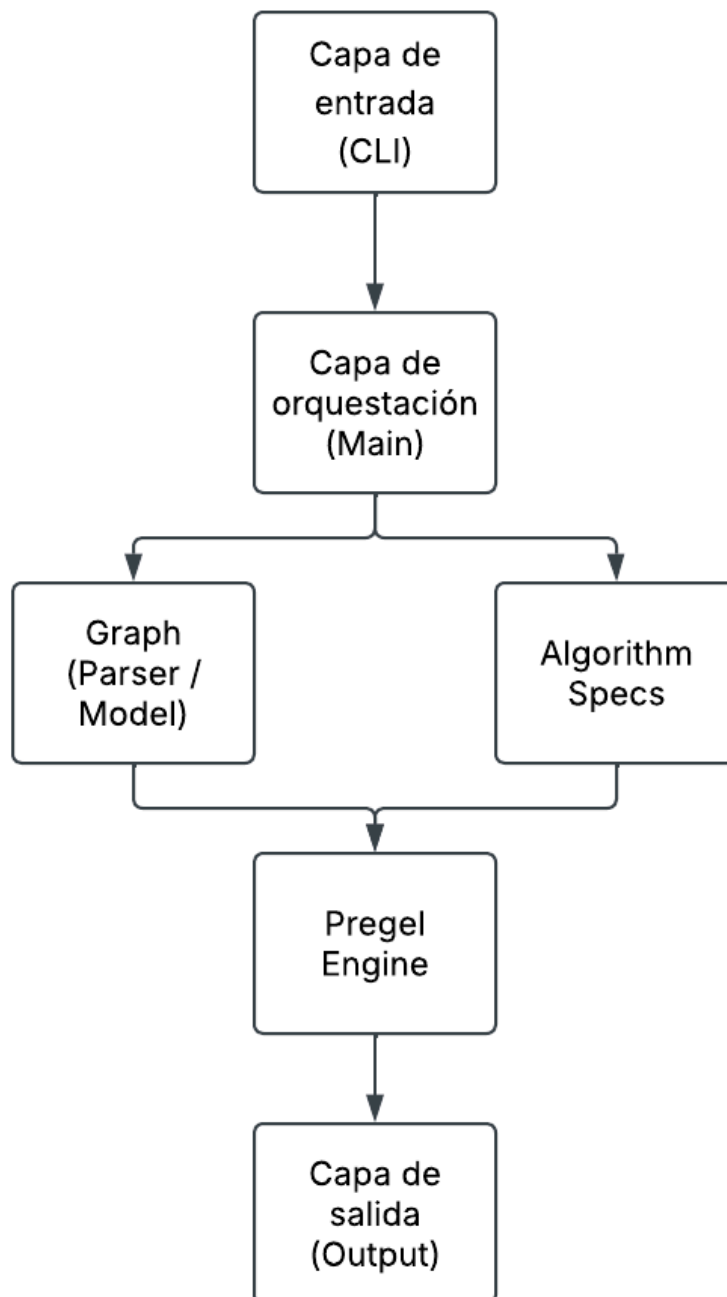
Las operaciones asociadas a entrada/salida, concurrencia y comunicación entre tareas se encuentran confinados a los módulos responsables de la ejecución del sistema (Main, Pregel.Engine, Pregel.Env, Pregel.Pool y Cli.Options).

Esta organización reduce el acoplamiento entre la lógica algorítmica y los aspectos operacionales de la aplicación, favoreciendo la modularidad, la reutilización y la capacidad de prueba de los componentes.

4. Detalles de Diseño

4.1. Arquitectura del Sistema

El sistema se organiza en capas con responsabilidades delimitadas. La capa de orquestación (Main) coordina un pipeline secuencial: validación de los argumentos de CLI, carga del grafo, resolución del algoritmo a utilizar, ejecución en Pregel y formateo de salida. El motor Pregel es agnóstico al algoritmo. Cada algoritmo implementado se expresa como un AlgorithmSpec, que el Main resuelve e inyecta en el motor. El flujo general se resume en la siguiente figura:



4.2. Estructura del Proyecto

Programa Principal (app/):

- Main.hs: Define el flujo principal de la aplicación. Parsea la CLI, válida origen y destino según el algoritmo elegido, carga el grafo, resuelve la especificación del algoritmo, invoca runPregel y construye las líneas de salida.
- AppError.hs: Unifica los errores de todas las capas en un único ADT y provee displayAppError para mensajes legibles en stderr.

Módulos:

- Graph (src/Graph): Dominio del grafo (tipos, validación, contexto de vértice y lectura de archivos)
 - Types.hs: Tipos centrales del dominio: NodeId, Distance, Weight, Edge, ValidGraph.
 - VertexContext.hs: Define VertexContext, una vista materializada por vértice (aristas salientes, pesos entrantes, grado, tamaño del grafo). Se precalcula una vez al inicio para que las funciones de actualización no consulten el grafo global en cada superstep.
 - Parser.hs: Parsea el formato de archivo de grafo. Expone loadGraphFile, describeGraph y validación de nodos de ejecución según el algoritmo.
 - ParseError.hs: ADT de errores de parseo con mensajes descriptivos (directiva desconocida, nodo fuera de rango, etc.).
- Algorithm (src/Algorithm): Lógica de negocio de cada algoritmo de grafos, expresado como especificaciones puras sobre el modelo Pregel.
 - Name.hs: Enumerado Algorithm que identifica los algoritmos disponibles y es usado en el registro de specs. Separado de Graph.Types porque no es una propiedad del grafo.
 - Types.hs: Define el contrato AlgorithmSpec state msg log (init, bootstrap, vertex update, extract, límite de supersteps, observabilidad) y el existencial SomeAlgorithmSpec para heterogeneidad en runtime. Incluye los constructores de conveniencia mkLabelSpec y mkRankSpec para familias de algoritmos recurrentes.
 - Spec.hs: Registro de algoritmos: resolveAlgorithm selecciona la spec según el flag CLI; validatePathSource y validatePathTarget exigen origen/destino solo para BFS y Bellman-Ford. Punto de extensión principal al añadir un algoritmo nuevo.
 - State.hs: Estados locales por vértice: PathState (distancia y predecesor), LabelState (etiqueta) y RankState (rank numérico).

- Messages.hs: Tipos de mensaje inter-vértice: DistanceMsg, LabelMsg y RankMsg.
- Result.hs: ADT de resultado final (PathFound, NoPath, Components, Rankings, NodeLabels) compartido por todos los algoritmos.
- Common.hs: Funciones compartidas: bootstrap e inicialización de caminos y etiquetas, runVertexUpdate (patrón genérico de actualización + emisión), extracción de resultados y utilidades de relajación (tryImproveDistance, tryRelabel).
- Log.hs: Entradas de log por superstep (PathLogEntry, LabelLogEntry, RankLogEntry), la typeclass MessageLog para registrar envíos de mensajes y la typeclass DescribeLogEntry con sus instancias para formateo en modo verbose.
- Observability.hs: Observadores de cambios de estado (pathObserver, labelObserver, rankObserver) usados en modo verbose.
- Error.hs: Errores semánticos de algoritmo (origen/destino faltante, nodo inexistente).
- Implementaciones concretas de algoritmos:
 - BFS.hs
 - BellmanFord.hs
 - PageRank.hs
 - ConnectedComponents.hs
 - LabelPropagation.hs
- Pregel (src/Pregel): Motor Pregel genérico, independiente del algoritmo concreto.
 - Types.hs: RunConfig, PregelRun, SuperstepLog, VertexStepResult y alias VertexStates / MessageQueues.
 - Engine.hs: Punto de entrada runPregel: inicializa el entorno STM, aplica bootstrap, ejecuta el bucle de supersteps con compute concurrente y finaliza con extracción de resultado.
 - Loop.hs: loopRunner, bucle de supersteps parametrizado en una mónada genérica m (reutilizado por el motor secuencial creado para tests); finalizeRun invoca specExtractResult.
 - Superstep.hs: Lógica pura de un superstep: processVertex, mergeSuperstepOutcomes, detección de cambio de estado y early halt.
 - Env.hs: Entorno concurrente con una TQueue STM por vértice; deliverAll entrega mensajes atómicamente entre supersteps y activeVerticesSTM lista vértices con mensajes pendientes.

- Pool.hs: Pool de workers con `async/wait`; limita la concurrencia al mínimo entre `--threads` y el número de vértices activos.
- Error.hs: Errores internos del motor (contexto o cola de vértice ausente, fallo en extracción de resultado).
- Cli (src/Cli): Interfaz de línea de comandos.
 - Options.hs: Define y parsea los flags (`-g`, `-s`, `-t`, `-a`, `--threads`, `-v`) mediante `optparse-applicative`. Valida que el número de threads esté entre 1 y las capacidades del RTS.
 - Error.hs: Errores de CLI: algoritmo desconocido, threads fuera de rango, identificador de nodo inválido.
- Output (src/Output): Formateo de la salida hacia el usuario.
 - Trace.hs: Describe la ejecución Pregel: resumen de supersteps, advertencia si se alcanzó el límite, y trazas detalladas por paso en modo verbose (usa `DescribeLogEntry` de `Algorithm.Log`).
 - Result.hs: Formatea el resultado final según el algoritmo (camino, componentes, rankings, etiquetas).
- Util (src/Util): Utilidades compartidas sin dependencia de dominio.
 - Reading.hs: Lectura de enteros desde String (`readNonNegativeInt`, `readPositiveInt`) y `trim`. Usado por `Graph.Parser` y `Cli.Options`.

4.3. Modelo de Grafos

4.3.1 Definiciones de tipos

El módulo `Graph` modela la topología del problema: nodos, aristas, validación y la vista local que consume el motor Pregel. No incluye la elección de algoritmo ni parámetros de ejecución.

Si bien internamente se manejan como `Int`, los identificadores y magnitudes se modelan como `newtypes` para permitir mayores validaciones en `compile-time`

```
newtype NodeId = NodeId { unNodeId :: Int }
newtype Distance = Distance { unDistance :: Int }
newtype Weight = Weight { unWeight :: Int }
```

Las aristas se definen utilizando un vértice origen, un vértice destino, y un peso. Para simplificar las validaciones requeridas y el diseño de tipos, en lugar de permitir aristas sin peso, el peso de aristas se toma por defecto como 1.

```
data Edge = Edge
  { edgeFrom    :: NodeId
  , edgeTo      :: NodeId
  , edgeWeight  :: Weight
  }
```

En la representación del grafo se mantiene una lista de nodos, una lista de aristas, y una lista de adyacencia saliente

```
data Graph = Graph
  { gNodes :: [NodeId]
  , gEdges :: [Edge]
  , gAdj   :: Map NodeId [(NodeId, Weight)]
  }
```

El campo `gNodes` enumera todos los vértices. Se usa al inicializar Pregel y en cualquier recorrido que necesite “todos los nodos” sin deducirlos de las aristas (debido a que pueden existir grafos con nodos que no tengan ninguna arista, lo que fuerza a mantener su listado completo).

`gEdges` conserva la lista completa de aristas tal como se cargó. Resulta necesaria para reconstruir pesos entrantes (véase `incomingWeights` en `VertexContext`) y para depuración mediante `describeGraph`, que resume nodos, aristas y adyacencia.

`gAdj` es la lista de adyacencia saliente: por cada `NodeId`, los pares (destino, peso) alcanzables en un paso. Permite consultar vecinos en tiempo acotado por $O(\log n + \text{grado})$ mediante `neighbors`, en lugar de filtrar `gEdges` en cada `superstep`.

A fin de evitar posibles problemas con estados de dominio inválidos, `Graph` no se exporta desde el módulo público: el resto del programa solo recibe `ValidGraph`, de modo que no se pueda construir un grafo “a mano” saltándose la validación.

```
newtype ValidGraph = ValidGraph Graph
```

`ValidGraph` es un `newtype` de certificación (también llamado `smart constructor` o tipo marcado): envuelve un `Graph` solo después de comprobar las invariantes, usarlo asegura que el grafo recibido respeta las reglas del dominio.

```
buildGraph :: Int -> [Edge] -> Either GraphError ValidGraph
```

`buildGraph` es la única puerta hacia `ValidGraph` en la biblioteca. Ante datos inválidos devuelve `GraphError` en lugar de un grafo parcial.

La función `isValidNode :: ValidGraph -> NodeId -> Bool` comprueba si un identificador pertenece al rango del grafo. La CLI y el parser la usan al validar `--source` y `--target` contra el grafo ya cargado.

Separar `Graph` de `ValidGraph` evita propagar `Maybe` o `Either` en cada función del motor. Pregel y los algoritmos reciben `ValidGraph` en `RunConfig` y confían en esas invariantes sin volver a comprobarlas.

Las funciones de consulta sobre el grafo son puras y de solo lectura. El estado del grafo es inmutable durante la ejecución del programa, mientras que el resultado del modelo de Pregel se mantiene en una estructura separada.

4.3.2. Contexto de vértice

Durante cada `superstep`, `specVertexUpdate` necesita la vista local del vértice: vecinos salientes, pesos entrantes y tamaño del grafo. Consultar `ValidGraph` en cada llamada repetiría trabajo y acoplaría el cómputo al tipo global.

Por eso, antes del primer `superstep`, `buildVertexContexts` precalcula un mapa `VertexContexts = Map NodeId VertexContext`:

```
data VertexContext = VertexContext
  { vcNodeId      :: NodeId
  , vcOutEdges    :: [(NodeId, Weight)]
  , vcInWeights   :: Map NodeId Weight
  , vcNodeCount   :: Int
  }
```

`vcOutEdges` réplica la adyacencia saliente de ese vértice. `vcInWeights` recoge las aristas entrantes: se obtiene filtrando `gEdges` por `edgeTo == v` y conservando los `weight`. `vcNodeCount` guarda el número total de nodos `n`, constante del grafo que cada vértice puede usar directamente (por ejemplo en la amortiguación de PageRank, donde interviene el factor $1/n$).

Las funciones `outNeighbors`, `outDegree`, y `lookupIncomingWeight` exponen esta vista sin que el algoritmo conozca la representación interna de `Graph`.

El precálculo implica memoria adicional del orden de $O(V + E)$ al inicio, a cambio de acceso local eficiente durante los `supersteps` y funciones `specVertexUpdate` más simples: reciben un `VertexContext` ya resuelto y permanecen puras respecto a la topología global.

4.3.3. Información repetida en modelos

Varias estructuras del modelo guardan datos que, en abstracto, podrían obtenerse a partir de otras. Esto es una decisión de diseño a favor de la optimización del algoritmo: hay una fuente canónica (las aristas parseadas, materializadas en `gEdges`) y el resto son índices o proyecciones calculados una sola vez al construir el grafo o al preparar `VertexContext`.

Ninguna se actualiza en runtime; al ser inmutables, la redundancia no introduce riesgo de desincronización.

- `gEdges` y `gAdj` (aristas y pesos salientes): Cada arista de `gEdges` aporta un `edgeFrom`, un `edgeTo` y un `edgeWeight`; la misma información aparece en `gAdj` como entrada (`edgeTo`, `edgeWeight`) bajo la clave `edgeFrom`. Se repiten destino y peso de toda arista saliente. Se mantiene `gEdges` como lista global porque hacen falta recorridos lineales sobre todas las aristas: pesos entrantes, salida de depuración (`describeGraph`) y tests. `gAdj` es un índice invertido por origen para que `neighbors` responda en tiempo acotado por el grado del nodo sin barrer `gEdges` en cada consulta. En Pregel esa consulta ocurre muchas veces por superstep; el coste es memoria extra del orden de $O(E)$ a cambio de evitar filtros repetidos del orden de $O(E)$ por vértice activo.
- `vcOutEdges` y `gAdj`: Para cada vértice, `buildVertexContexts` copia exactamente lo que devolvería `neighbors graph nodeId`. Es la misma adyacencia saliente por tercera vez, ahora embebida en el contexto local. La motivación es de interfaz y rendimiento en el hot path: `specVertexUpdate` recibe un único registro con todo lo que necesita del entorno topológico y no depende de `ValidGraph` ni de un `Map` global. Así el cómputo por vértice queda desacoplado del tipo de grafo y evita búsquedas en el mapa de adyacencia dentro del bucle paralelo de workers.
- `vcNodeCount` y `nodeCount`: El entero `n` se almacena una vez en el grafo y otra vez en cada `VertexContext`. Cada copia per-vértice permite que las funciones puras de algoritmo usen el tamaño del grafo (por ejemplo, el factor $1/n$ en PageRank) sin recibir `RunConfig` ni el `ValidGraph` completo. Es redundancia de un valor constante pequeño a cambio de firmas más simples en el contrato `AlgorithmSpec`.
- `vcInWeights` y `gEdges`: Aquí no se replica una fila entera de `gEdges`, sino una vista distinta: solo aristas que llegan al vértice. `gAdj` indexa vecinos salientes; los pesos entrantes no están en `gAdj` y habría que escanear `gEdges` filtrando por `edgeTo` cada vez que un algoritmo los necesite. Algoritmos como Bellman-Ford y PageRank consultan predecesores ponderados en cada actualización local. Precalcular `vcInWeights` como `Map NodeId Weight` convierte ese escaneo en una construcción única al inicio.

En conjunto, el diseño prioriza consultas locales rápidas en el motor Pregel frente a una representación mínima en memoria. Una variante más austera podría conservar solo `gEdges` y derivar el resto bajo demanda o una sola vez en un único índice bidireccional, pero la implementación actual elige materializar las vistas que el perfil de acceso hace más frecuentes.

4.4. Modelo de Pregel y Manejo de Concurrencia

4.4.1 Definiciones de Tipos

El módulo Pregel implementa el motor del modelo. El modelo recibe un `AlgorithmSpec`, y ejecuta el ciclo de supersteps sin conocer la semántica de mensajes ni el estado. La topología es recibida mediante un `ValidGraph`, y la lógica propia de negocio vive en cada `Algorithm`.

El diseño del motor persigue tres ideas. Primero, separar fases de Pregel: el cómputo por vértice es puro; solo las colas de mensajes y la orquestación viven en IO y STM. Segundo, parametrizar el bucle (loopRunner) sobre una mónada abstracta para reutilizar la misma lógica de convergencia en producción (IO + async) y en tests (StateT secuencial). Tercero, estado global inmutable entre supersteps: el mapa de estados por vértice se pasa y se fusiona explícitamente, no hay mutación del grafo lógico durante el compute.

```
data RunConfig = RunConfig
  { rcGraph      :: ValidGraph
  , rcSource     :: Maybe NodeId
  , rcTarget     :: Maybe NodeId
  , rcThreads    :: Int
  , rcMaxSteps   :: Int
  , rcTrace      :: Bool
  }

data VertexStepResult state msg = VertexStepResult
  { vsrState      :: state
  , vsrOutgoing   :: [(NodeId, msg)]
  }

data SuperstepLog log = SuperstepLog
  { sslStep        :: Int
  , sslActiveVertices :: Int
  , sslMessagesSent  :: Int
  , sslEntries      :: [log]
  }

data PregelRun log = PregelRun
  { prSupersteps    :: Int
  , prLogs          :: [SuperstepLog log]
  , prResult        :: Result
  , prMaxStepsReached :: Bool
  }

type VertexStates state = Map NodeId state
```

RunConfig agrupa grafo, parámetros de CLI, límite de hilos, tope de supersteps (por algoritmo, vía specMaxSupersteps) y flag de traza.

VertexStates es la snapshot global de estados en un superstep, y VertexStepResult es la salida local de un vértice (estado nuevo, mensajes salientes).

PregelRun cierra la ejecución con el Result del algoritmo, logs opcionales y un indicador si se alcanzó rcMaxSteps.

4.4.2. Orquestación y Ciclo Principal

El punto de entrada del motor concurrente encadena las fases de preparación, bootstrap y ciclo de supersteps:

```
runPregel cfg spec = do
  let graph = rcGraph cfg
      contexts = buildVertexContexts graph
  env <- initEnv graph
  let initialState = initialVertexStates spec cfg graph
  deliverResult <- deliverAll env (specBootstrap spec cfg)
  ...
  loopRunner cfg (activeVerticesSTM env) (runConcurrentSuperstep ...)
  (deliverAll env) 0 initialState
```

Primero se construyen los `VertexContexts` (vista local por vértice, capítulo 4.3). `initEnv` crea una `TQueue STM` por cada nodo del grafo. `initialVertexStates` aplica `specInitState` a todos los vértices, produciendo el mapa inicial de estados.

A continuación ocurre el bootstrap: `specBootstrap` genera mensajes iniciales (por ejemplo la primera oleada de rank en PageRank) y `deliverAll` los deposita en las colas antes de entrar al bucle. Esto corresponde a la fase de “mensajes de arranque” previa al superstep 0 lógico.

El bucle propiamente dicho delega en `loopRunner`, pasando tres efectos concretos en IO: cómo obtener vértices activos (`activeVerticesSTM`), cómo ejecutar un superstep (`runConcurrentSuperstep` con `runPool` y `async`), y cómo entregar mensajes al siguiente paso (`deliverAll`).

Cada iteración del bucle materializa las tres fases del modelo Pregel. En la fase de compute, los vértices con mensajes pendientes se procesan en paralelo: cada worker, en IO, vacía su cola con `flushVertexQueue`, invoca `processVertex` (que a su vez llama a la función pura `specVertexUpdate`) y devuelve el par estado/mensajes al hilo principal. `runPool` lanza hasta `min(rcThreads, |activos|)` tareas `async` por lote y hace `wait` sobre el lote completo antes de continuar; eso implementa la barrera del superstep.

En la fase de exchange, si hubo cambio de estado (`ssStateChanged`), `deliverAll` escribe todos los mensajes salientes en las colas destino dentro de una transacción STM. Los workers del superstep actual ya terminaron; ningún compute concurrente lee colas mientras se escribe, lo que respeta la semántica “los mensajes enviados en el paso t se leen en el paso $t + 1$ ”.

La convergencia combina tres condiciones de parada. Si no hay vértices con colas no vacías, el grafo lógico está inactivo y el bucle termina. Si ningún vértice activo modificó su estado en el superstep, se aplica `early halt`: no tiene sentido entregar mensajes ni seguir. Si el contador de supersteps alcanza `rcMaxSteps`, el bucle se detiene y `prMaxStepsReached` queda en `True`.

La función `loopRunner` se separó para ser reutilizada en un motor de procesamiento secuencial, el cual es usado para facilitar pruebas y realizar comparaciones de resultados con el modelo concurrente.

4.4.3. Ciclo y Superstep

`loopRunner` parametriza la repetición sobre una mónada `m`, recibiendo tres operaciones: vértices activos, ejecución de un superstep y entrega de mensajes.

```
loopRunner ::  
  Monad m => RunConfig -> m [NodeId] -> (...) -> (...) ->  
  Int -> VertexStates state ->  
  m (Either PregelError (VertexStates state, [SuperstepLog log], Int, Bool))
```

`Pregel.Superstep` concentra la lógica pura: `processVertex` invoca `specVertexUpdate` y, si `rcTrace` está activo, recopila logs; `mergeSuperstepOutcomes` fusiona los resultados del paso en un nuevo mapa de estados, concatena mensajes salientes y calcula `ssStateChanged`. `finalizeRun` aplica `specExtractResult` para producir el `PregelRun`.

4.4.4. Concurrencia

En el sistema principal, `PregelEnv` asigna una `TQueue STM` por vértice. Antes de cada superstep, `activeVerticesSTM` lista nodos con mensajes; cada worker vacía su cola (`flushVertexQueue`), ejecuta `processVertex` y devuelve el resultado; al cerrar el lote, `deliverAll` publica los mensajes salientes en una transacción atómicamente. `runPool` lanza workers con `async` en lotes de tamaño `min(rcThreads, |activos|)` y sincroniza con `wait`.

Para tests, `SequentialEngine.runPregelSequential` ejecuta el mismo `loopRunner` sobre `StateT` (`MessageQueues msg`) (`Either PregelError`): las colas son un `Map NodeId [msg]` actualizado con `enqueueMessages`. `processActiveVertices` es compartido.

4.5. Modelo de Algoritmos y Extensibilidad

El módulo `Algorithm/` expresa cada algoritmo como una especificación pura que el motor `Pregel` ejecuta sin conocer su semántica. El contrato central es `AlgorithmSpec`; la topología llega vía `VertexContext` y los tipos genéricos `state`, `msg` y `log`. Añadir un algoritmo nuevo es implementar ese registro y registrarlo en CLI y `resolveAlgorithm`, sin tocar el motor.

4.5.1. Tipos de Dominio

`Algorithm.Name` distingue la operación CLI del grafo:

```
data Algorithm  
  = BFS | BellmanFord | PageRank  
  | ConnectedComponents | LabelPropagation
```

Los estados locales viven en `Algorithm.State` en tres familias: `PathState` (distancia y predecesor, para BFS y Bellman-Ford), `LabelState` (etiqueta propagada, para CC y LP) y `RankState` (valor de rank, para PageRank). Los mensajes correspondientes son `DistanceMsg`, `LabelMsg` y `RankMsg` en `Algorithm.Messages`. El motor los trata como valores opacos en colas; solo el algoritmo les da significado.

El resultado final se unifica en `Algorithm.Result`:

```
data Result
  = PathFound [NodeId] Distance
  | NoPath
  | Components [(NodeId, [NodeId])]
  | Rankings [(NodeId, Double)]
  | NodeLabels [(NodeId, NodeId)]
```

`specExtractResult` proyecta el mapa final de estados a uno de estos constructores; `Output/Result.hs` lo formatea para `stdout`.

El contrato que deben implementar los algoritmos se maneja mediante `AlgorithmSpec`:

```
data AlgorithmSpec state msg log = AlgorithmSpec
  { specInitState      :: NodeId -> RunConfig -> state
  , specDefaultState  :: state
  , specBootstrap      :: RunConfig -> [(NodeId, msg)]
  , specVertexUpdate   :: VertexContext -> state -> [msg] ->
    VertexStepResult state msg
  , specExtractResult  :: Map NodeId state -> RunConfig -> Either
    AlgorithmError Result
  , specMaxSupersteps  :: Int -> Int
  , specObserveStep    :: NodeId -> state -> state -> [(NodeId, msg)] ->
    [log]
  }
```

`specInitState` asigna el estado local de cada vértice antes del primer superstep. Recibe el id del vértice y el `RunConfig` (origen, destino, grafo). En algoritmos de camino solo el `--source` arranca con distancia conocida; en etiquetas, cada vértice suele comenzar con su propio id como etiqueta; en PageRank, con $1/n$.

`specDefaultState` es el estado que usa el motor cuando un vértice aún no tiene entrada en el mapa global (`Map.findWithDefault`). Cubre el caso de vértices que aparecen en el mapa sin haber pasado por `specInitState` de forma explícita.

`specBootstrap` devuelve la lista de mensajes iniciales (destino, msg) que `runPregel` entrega con `deliverAll` antes de entrar al bucle. Arranca la dinámica del algoritmo: por ejemplo la primera inundación desde el origen en BFS o la distribución inicial de rank en PageRank.

`specVertexUpdate` es la función de cómputo local y pura: dado el contexto topológico del vértice, su estado actual y los mensajes recibidos en este superstep, produce un `VertexStepResult` con el estado nuevo y los mensajes salientes hacia vecinos. Es el núcleo semántico del algoritmo; el motor no interpreta su contenido.

`specExtractResult` corre una vez al converger el bucle. Recibe el mapa final de estados y el `RunConfig`, y devuelve un `Result` o un `AlgorithmError` (por ejemplo si faltan origen/destino o el nodo destino no fue alcanzado).

`specMaxSupersteps` calcula el tope de supersteps a partir del tamaño del grafo.

`specObserveStep` genera entradas de log cuando cambia el estado de un vértice. Si no hay cambio relevante, devuelve lista vacía.

4.5.2. Lógica compartida

Si bien se priorizó permitir un alto nivel de customización en el spec de algoritmo, en la práctica varios algoritmos repiten el mismo esqueleto. `Algorithm.Common` y los constructores `mkLabelSpec` / `mkRankSpec` en `Algorithm.Types` extraen esa repetición para que cada módulo implemente principalmente la lógica distintiva de cada algoritmo.

4.6. Manejo de Errores

El sistema no usa excepciones para flujos de control de negocio. Cada capa define su propio ADT de error y los fallos se propagan como valores `Left` hasta el borde de la aplicación, donde `Main` los traduce a un mensaje en `stderr` y termina con `die`. Las excepciones de I/O (`IOException` al leer el archivo de grafo) se capturan en `loadGraphFile` con `try` y se convierten en `LoadGraphError`.

La validación de flags de CLI vive en `Cli.Error` (`CliError`): algoritmo desconocido, `--threads` fuera de rango o por encima de las capacidades RTS. Esos errores se detectan al parsear opciones, antes de cargar el grafo. El parseo del archivo y la validación de nodos de ejecución usan `ParseError`. `LoadGraphError` envuelve fallos de lectura o errores de parseo al cargar desde disco. La construcción del grafo en memoria tiene su propio `GraphError` (conteo de nodos no positivo, extremo de arista inválido), que el parser reexpone como `ParseError` cuando corresponde.

En la capa de algoritmo, `AlgorithmError` cubre precondiciones de ejecución: origen o destino faltante para BFS y Bellman-Ford, o nodo destino inexistente al extraer el resultado (`TargetNodeMissing`). La resolución del algoritmo (`resolveAlgorithm`) es total y no participa de este `Either`. Los errores internos del motor se agrupan en `PregelError`, que puede anidar un `AlgorithmError` vía `ResultExtraction`.

`AppError` unifica las capas que llegan a `Main` (`AppAlgorithm`, `AppParse`, `AppLoad`, `AppPregel`) y delega el formateo a cada `display*Error`. El pipeline completo se compone con `runExceptT` desde `parseOptions`:

```
runApp = runExceptT $ do
```

```
opts <- exceptTWith AppCli =<< liftIO parseOptions
...
```

Las funciones puras del núcleo (`parseGraphFile`, `buildGraph`, `processVertex`, `specExtractResult`) devuelven `Either` y nunca lanzan excepciones; los errores se combinan con `>=>`, `fmap` o `first`. `liftIO` delimita dónde entran efectos reales (lectura de archivo, STM, `async`). Un único `die` (`displayAppError err`) en `main` cierra cualquier `Left`.

4.7. Manejo de Trazas

Durante el desarrollo resultó necesario observar la evolución de cada ejecución para comprender el comportamiento de los algoritmos, validar la propagación de mensajes y facilitar la depuración del motor Pregel. Sin embargo, esta información no forma parte del resultado del algoritmo, sino de un mecanismo auxiliar de observabilidad. Por este motivo, el sistema separa explícitamente el registro de eventos de la lógica de procesamiento y permite habilitarlo únicamente cuando el usuario solicita una ejecución detallada.

Dado que esta información resulta útil principalmente durante el desarrollo y la depuración, la observabilidad se implementó como una característica opcional. La configuración de ejecución (`RunConfig`) incorpora el indicador `rcTrace`, habilitado desde la línea de comandos mediante `-v` o `--verbose`. Cuando esta opción no está activa, la aplicación presenta únicamente el resumen de la ejecución y el resultado final; al habilitarla, el motor registra información detallada de cada `superstep`.

El registro de eventos se integra con el procesamiento de cada `superstep` sin modificar la lógica de los algoritmos. Tras ejecutar `specVertexUpdate`, el motor consulta opcionalmente `specObserveStep`, que permite registrar los cambios relevantes de estado producidos durante la actualización de un vértice. De forma análoga, el envío de mensajes se instrumenta mediante la `typeclass` `MessageLog`, encargada de convertir cada mensaje emitido en una entrada de traza. Al finalizar cada `superstep`, el motor reúne todas las entradas generadas junto con información agregada (como la cantidad de vértices activos y mensajes enviados) en un `SuperstepLog`, que posteriormente es utilizado por `describeRun` para construir la salida detallada.

Cada familia de algoritmos define su propio tipo de entrada de log (`PathLogEntry` para BFS y Bellman-Ford, `LabelLogEntry` para Componentes Conexas y Label Propagation, y `RankLogEntry` para PageRank), registrando únicamente los eventos relevantes para ese dominio. La `typeclass` `DescribeLogEntry` encapsula el formateo de estas entradas y define un criterio de ordenamiento estable para la salida. Mantener esta responsabilidad junto al tipo de log preserva la separación entre el motor Pregel y la capa de presentación, evitando que `SomeAlgorithmSpec` dependa de módulos dedicados exclusivamente al formateo.

Con `verbose = False`, `describeRun` informa la cantidad de `supersteps` ejecutados, advierte si se alcanzó el límite configurado (`rcMaxSteps`) y presenta el resultado mediante `describeResult`. Cuando `verbose = True`, incorpora además un bloque por `superstep` con estadísticas generales y el detalle de los eventos registrados.

Esta organización también simplifica la incorporación de nuevos algoritmos. Cada implementación puede definir sus propios tipos de eventos, proporcionar las instancias correspondientes de `MessageLog` y `DescribeLogEntry`, e implementar `specObserveStep` cuando resulte útil registrar cambios de estado. Si un algoritmo no requiere trazas específicas, basta con devolver una lista vacía de eventos, sin necesidad de modificar el motor ni la infraestructura de presentación.

4.7.1 Alternativas Consideradas

Existen distintas estrategias para incorporar observabilidad en aplicaciones funcionales. Una alternativa habitual en Haskell consiste en utilizar una mónada de anotación, como `Writer` o `WriterT [LogEntry]`, de modo que cada paso del cómputo acumule eventos mediante `tell`. Otra posibilidad consiste en registrar las trazas directamente en IO o STM durante la ejecución concurrente de los `supersteps`.

En este proyecto se optó por una estrategia diferente, basada en la acumulación explícita de listas (`[log]`) durante el procesamiento de cada vértice. De esta manera, `specVertexUpdate` permanece completamente pura y libre de dependencias con mecanismos de logging, mientras que la observabilidad se incorpora posteriormente mediante `specObserveStep` y `MessageLog`. El indicador `rcTrace` únicamente controla la recolección de esta información: cuando la traza está deshabilitada, el motor omite la generación de entradas de log sin modificar el flujo de ejecución del algoritmo.

Esta decisión aporta varias ventajas para la arquitectura del sistema. En primer lugar, la lógica algorítmica conserva la transparencia referencial y puede verificarse mediante pruebas unitarias sin necesidad de simular efectos de logging. En segundo lugar, el mismo código de procesamiento puede reutilizarse tanto en el motor secuencial como en el concurrente, permitiendo que ambos produzcan exactamente las mismas trazas cuando estas se encuentran habilitadas. Esto hace posible comparar ejecuciones completas, incluyendo el contenido de `prLogs`, y utilizar el motor secuencial como referencia para validar el comportamiento del concurrente. Finalmente, la responsabilidad de presentar las trazas queda confinada al borde de la aplicación (`describeRun`), preservando la separación entre el dominio, el motor de ejecución y la capa de presentación.

Como contrapartida, esta estrategia requiere transportar explícitamente las entradas de log durante la ejecución, además de parametrizar por el tipo log estructuras como `AlgorithmSpec` y `PregelRun`. Este incremento en la cantidad de datos intercambiados se consideró un costo razonable frente a las ventajas obtenidas en términos de modularidad, testabilidad y claridad en el manejo de los efectos.

5. Metodología de Pruebas

La validación del sistema se realizó combinando distintos niveles de pruebas, aprovechando la separación entre el núcleo funcional del modelo y la infraestructura de ejecución concurrente. Las funciones puras del dominio se verifican mediante tests unitarios, mientras que el comportamiento completo del sistema se valida mediante pruebas de integración que ejecutan el motor Pregel sobre grafos de ejemplo. Complementariamente, se utilizan pruebas basadas en propiedades (QuickCheck) para comprobar invariantes generales del modelo sobre grafos generados automáticamente, y pruebas de presentación para asegurar que la salida producida por la aplicación permanezca consistente.

Esta organización permite verificar el sistema desde diferentes perspectivas. Las pruebas unitarias detectan errores locales en funciones aisladas; las de integración validan la interacción entre los distintos módulos; las pruebas de propiedades ejercitan el sistema sobre una mayor variedad de entradas que las contempladas por los fixtures; y las pruebas de salida aseguran que el comportamiento observable por el usuario permanezca estable frente a modificaciones internas.

El soporte común de la suite se concentra en módulos auxiliares encargados de administrar los grafos de prueba, construir configuraciones de ejecución y definir oráculos reutilizables para la validación de caminos, rankings y resultados de los distintos algoritmos.

5.1. Motor Secuencial Como Referencia de Validación

Una decisión importante del diseño de la estrategia de pruebas fue desarrollar un segundo motor de ejecución completamente secuencial. A diferencia de una implementación independiente del modelo Pregel, este motor reutiliza exactamente el mismo ciclo principal (loopRunner) que la versión concurrente. Ambos comparten la lógica de convergencia, la detección de vértices activos, el procesamiento por supersteps y la extracción del resultado final, diferenciándose únicamente en la infraestructura utilizada para intercambiar mensajes entre vértices.

En el motor concurrente, la comunicación se realiza mediante colas TQueue administradas por STM y el procesamiento de los vértices se distribuye entre múltiples workers utilizando async. En cambio, el motor secuencial reemplaza estos mecanismos por estructuras de datos puramente funcionales y ejecuta los vértices uno tras otro dentro de una StateT.

Gracias a esta arquitectura, el motor secuencial puede utilizarse como una referencia de corrección para el concurrente. Las pruebas ejecutan ambos motores sobre la misma configuración y verifican que producen el mismo resultado, la misma cantidad de supersteps y el mismo comportamiento observable. De esta manera, cualquier discrepancia puede atribuirse con alta probabilidad a la infraestructura de concurrencia o al intercambio de mensajes, y no a la lógica propia de los algoritmos implementados.

Asimismo, el carácter determinista del motor secuencial permite utilizarlo como base para las pruebas basadas en propiedades, garantizando que una misma configuración produzca siempre el mismo resultado.

5.2. Verificación Basada en Propiedades

Además de los casos concretos incluidos en los tests unitarios y de integración, el proyecto incorpora pruebas basadas en propiedades mediante QuickCheck. En este enfoque no se verifica únicamente el resultado esperado para un conjunto reducido de entradas, sino que se formulan invariantes generales que deben cumplirse para una amplia variedad de grafos generados automáticamente.

Las propiedades comprobadas abarcan distintos aspectos del sistema. En primer lugar, se valida la corrección de los algoritmos comparando sus resultados con oráculos independientes, como el cálculo de caminos mínimos o la verificación de la estabilidad alcanzada por Label Propagation. En segundo lugar, se comprueba que los motores concurrente y secuencial permanezcan observacionalmente equivalentes sobre los grafos generados, extendiendo esta validación más allá de los casos representativos incluidos en el repositorio.

Por otra parte, las propiedades también ejercitan el núcleo funcional del sistema verificando invariantes sobre las operaciones puras compartidas por distintos algoritmos, como la monotonía de la relajación de distancias, la idempotencia de las operaciones de etiquetado y la reconstrucción consistente de caminos. Finalmente, se incluyen verificaciones relacionadas con la carga y validación de grafos, asegurando que los archivos de ejemplo respeten las restricciones del formato definido por la aplicación.

El uso de QuickCheck complementa las pruebas tradicionales al proporcionar una cobertura significativamente mayor del espacio de entradas posibles sin necesidad de escribir manualmente un caso de prueba para cada configuración.

5.3 Validación de la salida de la aplicación

La estrategia de pruebas no se limita a verificar la corrección del procesamiento interno del modelo Pregel. También se comprueba la estabilidad de la salida producida por la aplicación, tanto en el modo normal de ejecución como en el modo detallado (--verbose).

Estas pruebas validan que el resumen de la ejecución refleje correctamente la cantidad de supersteps realizados, la detección del límite máximo de iteraciones y el resultado final obtenido por cada algoritmo. Del mismo modo, se verifica que el modo detallado incorpore la información adicional correspondiente a cada superstep únicamente cuando dicha opción se encuentra habilitada.

Asimismo, se comprueba el formateo de cada uno de los tipos de resultados soportados por la aplicación, asegurando que los ejemplos presentados en el capítulo de ejecución coincidan con la salida generada por el sistema.

La separación entre la lógica algorítmica y el módulo encargado del formateo hace posible validar esta capa de manera independiente. De esta forma, cambios en la presentación de resultados pueden detectarse mediante la suite de pruebas sin afectar las verificaciones sobre el funcionamiento interno del motor Pregel.

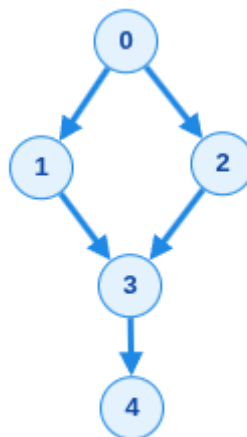
6. Ejemplos de ejecución

El repositorio cuenta con archivos de ejemplo que definen grafos en los que probar el sistema. A continuación se describen los ejemplos y resultados esperados, junto con el comando requerido para ejecutarlos.

6.1 BFS

6.1.1 Camino mínimo

El grafo de prueba es grafo-simple.txt, que describe a:



Se busca un camino mínimo entre los vértices 0 y 4. Para eso se debe ejecutar:

```
cabal run graphaskell -- -g examples/grafos-simple.txt -s 0 -t 4 -a BFS
```

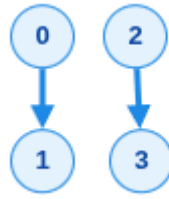
El resultado es

```
Result: path found
Distance: 3
Path:      [0,1,3,4]
```

Nota: El camino [0,2,3,4] también tiene distancia 3; el algoritmo podría devolver cualquiera de los caminos óptimos.

6.1.2 Sin camino entre vértices

El grafo de prueba es grafo-disconnected.txt, que describe a:



Se busca el camino mínimo entre los vértices 0 y 3 (el cual no existe). Si se ejecuta

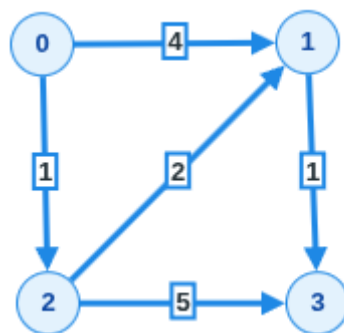
```
cabal run graphaskell -- -g examples/grafos-disconnected.txt -s 0 -t 3 -a
BFS
```

El resultado es:

```
Result: no path between source and target
```

6.2. Bellman-Ford

El grafo de prueba es grafo-weighted.txt, que describe a:



Se busca el camino mínimo entre 0 y 3, ponderando los pesos de las aristas. Para esto se debe ejecutar:

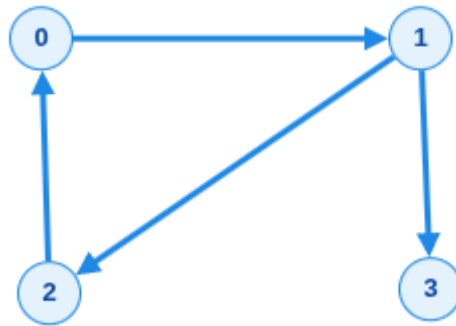
```
cabal run graphaskell -- -g examples/grafos-weighted.txt -s 0 -t 3 -a
BELLMANFORD
```

El resultado es:

```
Result: path found
Distance: 4
Path:      [0,2,1,3]
```

6.3 PageRank

El grafo de prueba es grafo-pagerank.txt, que describe a:



Se busca obtener una aproximación de la importancia de cada nodo, mediante la fórmula del algoritmo de PageRank. Para ejecutarlo, se debe realizar:

```
cabal run graphaskell -- -g examples/grafos-pagerank.txt -a PAGERANK
```

El resultado obtenido es:

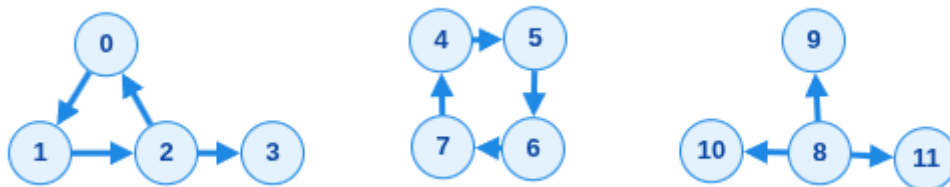
```
Result: PageRank
node 0: 0.12002296283541904
node 1: 0.13951951841010618
node 2: 0.09679579532429514
node 3: 0.09679579532429514
```

Nota: En la implementación de PageRank realizada, se fija el factor de amortiguación a 0.85 y la tolerancia a $1e-9$. El número máximo de supersteps se limita a n^2 , siendo n la cantidad de vértices del grafo.

El nodo 1 obtiene el mayor PageRank porque concentra todo el flujo procedente del nodo 0 (único sucesor de 0 en el ciclo $[0,1,2,0]$), mientras que el nodo 2 solo recibe una parte del rank del 1 al repartirse entre los nodos 2 y 3.

6.4 Componentes Conexas

Se usa el grafo grafos-componentes.txt



Se quieren obtener las componentes conexas del grafo, es decir, todos los subgrafos en donde exista un camino entre todos los nodos de dicho subgrafo. Para eso se debe ejecutar:

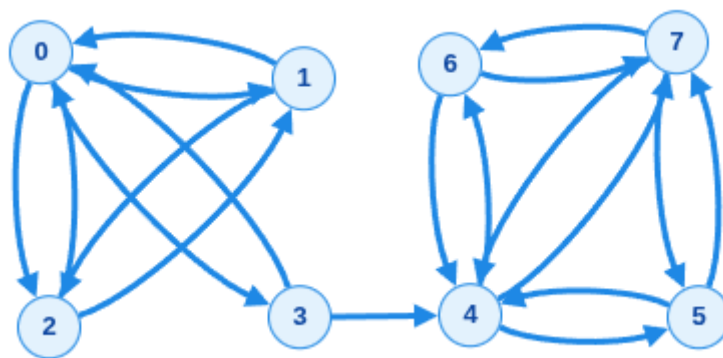
```
cabal run graphaskell -- -g examples/grafos-componentes.txt -a CC
```

El resultado es:

```
Result: connected components
component 0: [0,1,2,3]
component 4: [4,5,6,7]
component 8: [8,9,10,11]
```

6.5 Label Propagation

Se usa el grafo grafo-lp-comunidades.txt



Nótese que el grafo tiene un único componente conexo, pero consiste en 2 grupos densamente conectados unidos por un único puente en [3,4]. Lo que se busca obtener son los grupos fuertemente conectados del grafo. Para eso se ejecuta

```
cabal run graphaskell -- -g examples/graf0-lp-comunidades.txt -a LP
```

Y el resultado obtenido es:

```
Result: label propagation
node 0 -> label 0
node 1 -> label 0
node 2 -> label 0
node 3 -> label 0
node 4 -> label 3
node 5 -> label 3
node 6 -> label 3
node 7 -> label 3
```

7. Cambios Realizados Respecto a la Propuesta Inicial y Limitaciones de Diseño

En la propuesta inicial entregada, se planteó la implementación del modelo de Pregel con 3 algoritmos concretos: BFS, DFS y Dijkstra. Desde la concepción inicial del proyecto, el alcance fue reevaluado y se decidió realizar un cambio en el proyecto para descartar la implementación de DFS y Dijkstra, y en su lugar realizar la implementación de otros algoritmos que encajan mejor con el modelo de Pregel.

Dada la forma de procesar vértices de Pregel, los algoritmos de DFS y Dijkstra no tienen implementaciones eficientes en el modelo. En particular, DFS requiere seguir un camino único de nodos, desaprovechando el procesamiento concurrente que brinda Pregel, y en la práctica suele reemplazarse por BFS para realizar búsquedas si se desea concurrencia.

Algo similar sucede con Dijkstra. En este caso, se implementó Bellman-Ford, que es un algoritmo que también se usa para obtener distancias en un grafo ponderado de forma similar a Dijkstra, aunque cuenta con la limitación de solo poder calcular la distancia mínima entre un par (origen, destino) específico por ejecución, a diferencia de Dijkstra que calcula la matriz completa de destinos y orígenes.

Se implementaron, además, otros algoritmos que aprovechan bien el modelo concurrente (PageRank, CC, LP), a fin de demostrar casos de uso reales del modelo. Y si bien no fue algo planteado en la propuesta inicial, se aprovechó la oportunidad para diseñar el modelo de Pregel de forma extensible, permitiendo implementar otros algoritmos en el futuro de forma sencilla.

La principal limitación de este sistema radica en que no procesa los grafos de forma distribuida, característica que está permitida por el modelo de Pregel teórico. Solamente usa concurrencia en los threads de una misma computadora. Esta decisión se tomó para limitar el alcance del proyecto, y evitar las complejidades asociadas con el manejo de mensajes mediante red y tolerancia a fallos.

Otra limitación es que el sistema solo modela grafos dirigidos, no permitiendo definir de forma directa aristas bidireccionales. Esta decisión de implementación es concordante con otros frameworks de grafos, pero impone unas condiciones a seguir para tanto la definición de grafos como de algoritmos, forzando a replicar grafos no dirigidos mediante aristas entrantes o duplicación de aristas en la definición de grafos.

7.1. Aclaración Sobre Uso de Inteligencia Artificial en el Desarrollo

Con el objetivo de optimizar el tiempo de desarrollo, se empleó asistencia de herramientas de Inteligencia Artificial para el proyecto. En particular, se utilizó de las siguientes formas:

- **Parseo de argumentos de CLI:** Se utilizó para investigar sobre alternativas de bibliotecas para el manejo de argumentos, optando por Optparse-Applicative. Luego, se generó una primera versión del código del parser con los argumentos y validaciones especificadas.
- **Código de tests:** Para simplificar la escritura de los tests, los cuales usualmente requieren una cantidad alta de código para setup y validación, se generó una

primera versión de la suite de tests a partir de una especificación en lenguaje natural y luego se iteró sobre el código generado.

- Validaciones y sugerencias sobre el código escrito: Dado que el desarrollo fue escrito por una única persona, no era factible obtener una segunda opinión sobre el código escrito, corriendo el riesgo de cometer errores de diseño o realizar decisiones mejorables en el código. A fin de intentar mitigar esto, se realizaban consultas sobre fragmentos del código escrito, intentando obtener oportunidades de mejora sobre este. Se tuvo especial cuidado con las respuestas obtenidas en esto, dado que la IA no cuenta con el contexto completo del sistema, únicamente con las partes compartidas.
- Corrección del informe: Se enviaron fragmentos del texto escrito en este informe, a fin de detectar problemas de ortografía y gramática, además de sugerir redacciones alternativas en párrafos para facilitar la lectura.

En todos los casos, se tomó precaución de revisar y realizar correcciones sobre el contenido generado y sugerido, pasando por una revisión manual para asegurar que no surjan problemas a partir de la delegación del trabajo en una herramienta estadística con tendencia a producir errores.

8. Conclusiones y Trabajo Futuro

El objetivo de este trabajo fue diseñar e implementar una aplicación en Haskell que permitiera estudiar la aplicación de los principios de la programación funcional en un problema de procesamiento concurrente sobre grafos.

La implementación permitió comprobar que es posible expresar este modelo de forma natural mediante funciones puras para la lógica de cómputo y encapsular los efectos asociados a concurrencia, comunicación y entrada/salida en componentes bien delimitados. El uso de STM y async facilitó la coordinación segura entre los distintos workers, mientras que la separación entre el motor Pregel y las especificaciones de los algoritmos permitió mantener un alto grado de modularidad y reutilización.

Asimismo, la incorporación de algoritmos de distinta naturaleza, como BFS, Bellman-Ford, PageRank, Componentes Conexas y Label Propagation, permitió validar que una misma infraestructura de ejecución puede reutilizarse para resolver problemas diversos únicamente modificando la función de actualización de cada vértice. Esto demuestra que la abstracción propuesta resulta suficientemente general para representar distintos patrones de procesamiento sobre grafos.

Desde el punto de vista de la programación funcional, el proyecto también permitió aplicar de manera integrada conceptos abordados durante la cursada, tales como el modelado mediante tipos algebraicos, el uso de mónadas para el manejo de efectos, la transparencia referencial en el núcleo del sistema y la composición de funciones puras con componentes concurrentes.

Como trabajo futuro, existen diversas líneas de evolución posibles. En primer lugar, el sistema podría ampliarse incorporando nuevos algoritmos compatibles con el modelo Pregel, como Single Source Shortest Path con detección de ciclos negativos, algoritmos de flujo o variantes adicionales de detección de comunidades.

Otra línea de trabajo consiste en realizar una evaluación experimental más exhaustiva del motor desarrollado. Esto incluiría medir tiempos de ejecución sobre grafos de distintos tamaños, analizar el impacto de la cantidad de threads utilizados y comparar el comportamiento del motor concurrente respecto de la implementación secuencial utilizada para pruebas.

Otra mejora sería el manejo nativo de grafos no dirigidos. Como primera iteración de esto, se podría incluir una directiva en la definición de grafos que indique si el grafo está o no dirigido, y ajustar la representación interna de los vértices acorde a eso.

Finalmente, una evolución natural del proyecto sería extender el motor hacia un entorno distribuido, reemplazando la comunicación local entre threads por mecanismos de comunicación entre procesos o nodos de una red. Si bien esto implicaría desafíos adicionales relacionados con particionamiento de grafos, tolerancia a fallos y sincronización distribuida, permitiría acercar la implementación al modelo Pregel original y evaluar su comportamiento sobre volúmenes de datos considerablemente mayores.

9. Bibliografía

- [1] Pregel: A System for Large-Scale Graph Processing. (2010). Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, & Grzegorz Czajkowski. En *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data* (pp. 135–146). ACM.
- [2] Graph Algorithms with a Functional Flavour. John Launchbury. (1995). En *Graph-Theoretic Concepts in Computer Science* (Lecture Notes in Computer Science, Vol. 925). Springer.
- [3] Functional Programming with Graphs. Martin Erwig. (1997).
- [4] Haskell in Depth. Vitaly Bragilevsky. (2021). Manning Publications.